

Compiler Construction - CLamb

Project Proposal

1. Team

Aman Khan - 29029

2. Language Name

CLamb

3. Language Idea / Motivation

The language is a reduced version of C. Its purpose is to serve as a medium to create a compiler and understand how a language is constructed with its rules, defined types, and specifications.

4. Target Language / Inspiration

The inspiration for this language is largely taken from C. It is largely a subset of C with one unique addition of my own.

5. Unique Features

C does not have lambda functions. They are a very important feature in many modern languages. This language will have lambda functions. Since the implementation of proper lambdas that capture variables from their environment is quite a difficult task (as they require the usage and implementation of “closures”), the goal is to stick to non-capturing lambdas.

Lambda functions will be defined by the keyword “lamb” instead of the C++ standard “`λ`”.

This is easy and readable.

Since the lambda function is non-capturing, no dynamic allocation is needed, and hence the lambda function can stay on the stack.

NOTE: The lambda function feature can be changed from non-capturing to capturing with dynamic allocation if the scope seems too restricted for the instructor.

6. Planned Language Features

6.1.Data Types:

Primitive Data Types: int, float, char, bool, string

Type Conversions: Strictly strongly typed. Only explicit type casting is allowed (e.g. int(3.14)). Implicit type coercion is not supported to keep type checking straightforward.

6.2.Variable Declarations:

We will stick to explicit static declarations in this language. It will remain easy to read and understand.

```
<Type> <Identifier> = <Expression>  
auto <identifier> = <Expression>
```

6.3.Arithmetic & Relational Operators:

Arithmetic Operators: +, -, *, /, and %

Relational Operators: <, >, <=, >=, ==, !=

Logical Operators: AND, OR, NOT

6.4.Arrays:

Fixed-size 1- and 2-dimensional arrays. The size of the array must be defined explicitly. I wanted to implement a multi-dimensional array but also not make the project too difficult, so I capped the dimensions to 2.

The array will be row-major. The initialization will be as follows;

```
Int array[4];  
Int array2d[4][7];
```

Any assignment from there on will be done using indexing for both 1D and 2D array.

6.5.Pointers & References:

For reference: & (address-of operator)

For pointers: * (Dereference operator)

Explicit pointers will be implemented.

Dynamic memory allocation/deallocation on the heap via built-in malloc(bytes) and free(ptr) functions.

6.6.Control Flow:

Conditions: if /else will be used.

```
If ( condition ) { ... } else { ... }
```

Loops: for and while will be implemented.

```
for (init; condition; increment) { ... }  
while (init; condition) { ... }
```

6.7.Functions:

Will follow the C standard format. User-defined functions with parameter passing and return values (including void).

6.8.Lambdas:

Will be defined using the keyword “lamb”. The structure is as follows:

```
lamb(parameters = <type> <identifier>){...}
```

7. Sample Programs

```
int main() {  
    // Assigning a non-capturing lambda to a variable  
    auto add = lamb(int a, int b) {  
        return a + b;  
    };  
  
    int result = add(10, 20);  
    return 0;  
}
```

```

int main() {
    int arr[3];
    arr[0] = 5;
    arr[1] = 10;

    int* ptr = &arr[0];

    if (*ptr < 10) {
        for (int i = 0; i < 2; i = i + 1) {
            arr[i] = arr[i] * 2;
        }
    }
    return 0;
}

```

8. Initial Lexical Specifications

These are just examples of what will be formalized later

Token Category	Token Class Members Examples
Keywords	int, float, char, bool, string, if, else, for, while, return, lamb, null, void, print, malloc, free, auto
Identifiers	main, ptr, result, a, answer69, variable_name
Literals	10, 20, 3.14, 'a', "hello", true, false
Arithmetic Operators	+, -, *, /, %
Relational Operators	<, >, <=, >=, ==, !=
Logical Operators	AND, OR, NOT
Assignment / Address	=, &, *
Punctuators	(,), {, }, [,], :, ,

9. Potential Risks

Handling “lamb” expressions requires generating a unique global function entry point at compile-time and verifying function pointer type compatibility during assignment or function calls.

Designing LL(1) / LALR parser rules to avoid shift/reduce conflicts when evaluating inline “lamb” expressions passed inside parameter lists.

Keeping pointer semantics strictly limited to * and & operations without full pointer arithmetic to keep stack allocation and code generation manageable.

Generating intermediate code for 2D array offset calculations ($\text{Row} * \text{NumColumns} + \text{Col}$) during semantic analysis and code generation.